

Skills platform: public audit & automation architecture mapping

Two decisions in one document. **Part 1** audits all 34 skills in the repo and rules on which belong in the public npm package (`@nexor/skills`) now publishing on every merge to `main` . **Part 2** is the new `automation-architecture` skill's mapping: how a client use case becomes an exact configuration of agents, transfers, and the five automation function types — grounded in the real backend semantics of `nexor-node-api` .

27

KEEP PUBLIC

2

KEEP, NEEDS EDIT

4

WITHDRAW TO
INTERNAL

1

WITHHELD (3RD-
PARTY)

+1

NEW SKILL
CREATED

Why four withdrawals

The repo's own contribution rule — “no customer names, no personal data, no production statistics, no credentials, no internal file paths” — was applied to **content**. This audit applies it to **capability**: a skill is withdrawn when publishing it hands a stranger operational knowledge of Nexor's production systems, even with every name scrubbed.

SKILL	VERDICT	REASON IT CANNOT STAY PUBLIC
query-catalog	WITHDRAW	455 lines of canonical SQL over the production schema — a complete map of table and column names. The transferable idea (a canonical-query catalog per metric) survives as one paragraph; the content is 95% Nexor's database.
quick-meeting	WITHDRAW	Documents that the production booking endpoint accepts internal writes with only a client-id header and no user token. Even with the hostname redacted, that discloses the auth model of a prod write path.
webchat-incident	WITHDRAW	Names the abuse-defense blind spots — the fingerprint detector requires letter+digit suffixes, and the honeypot layer that catches 100% of historical attacks “costs one line to evade.” Publishing your defenses' gaps is a gift to the next attacker.
nexor-onboard	WITHDRAW	An internal operator tool: its scripts read the production DB with a service-role key that bypasses RLS, and its outputs are client lists. The public funnel concept (T0→T4) is already summarized in the skill prose; the tool itself has no public user.

Mechanics already in place. Withdrawal is one line per skill in `package.json` → `nexorSkills.unpublished` plus a matching `files` negation. The CI gate (`scripts/prepublish-check.mjs`) then hard-fails any build where one of these files would reach the tarball — the same mechanism that already withholds `agent-browser` . The skills stay in the repo for internal agents.

Verdict on every skill (1 of 2)

Groups mirror the plugin manifest. "Keep" = publish in `@nexor/skills`; "Internal" = repo-only, excluded from the tarball.

SKILL	VERDICT	RATIONALE
PROMPT ENGINEERING		
claude-prompt-fundamentals	KEEP	Canonical prompt doctrine. Fully transferable.
prompt-rewrite-checklist	KEEP	Pre-ship checklist; no coupling to Nexor internals.
xml-tag-discipline	KEEP	Editing tagged prompts a runtime reads by name — generic pattern.
voice-scripting-v3	KEEP	ElevenLabs-v3 TTS scripting rules; strong public content.
nexor-prompt-style	KEEP	House philosophy ("rules-light, processor-heavy"); it is the brand's craft on display.
AGENT PLAYBOOKS		
add-booking-provider	KEEP	Already rewritten vendor-agnostic; the flagship deep skill.
appointment-reminders-playbook	KEEP	Autonomous single-approval config flow; teaches the pattern, names only product-level tools.
whatsapp-template-marketer	KEEP	Infer-don't-interrogate drafting playbook.
meta-whatsapp-templates-spec	KEEP	Encodes Meta's public spec; zero Nexor coupling.
how-to-update-all-workflow	KEEP	Lockstep domain-change rewrite across 8 surfaces; transferable discipline.
how-to-manage-executives	KEEP	Rep-assignment + NL routing-rule playbook, product-level vocabulary only.
data-analysis-playbook	EDIT	Split: keep the text-to-SQL hygiene rules public; move the exact production column tables into <code>internal/</code> (tarball-excluded). Today it is a partial schema map.
query-catalog	INTERNAL	Full production schema exposure — see page 1.
ideal-lead-profile	KEEP	Engagement-depth lead scoring concept.
how-to-move-leads	EDIT	Keep — but scrub the production failure-rate figures (38.8% / 33.1%) from the description; they violate the repo's "no production statistics" rule.
nexor-add-language	KEEP	Reframed as a worked map ("what transfers is the shape of the failures"). Silent-failure taxonomy is the value.
quick-meeting	INTERNAL	Prod write-path auth disclosure — see page 1.
automation-architecture	NEW	Created in this pass — Part 2 of this document is its mapping. Public: it documents the product's configuration surface, which the public MCP already exposes to clients.

Verdict on every skill (2 of 2)

SKILL	VERDICT	RATIONALE
ENGINEERING DISCIPLINE		
engineering-guardrails	KEEP	Pre-flight/pre-ship review for LLM systems; incidents genericized to lessons.
generate-flow-eval	KEEP	Three-layer eval methodology (unit + golden + E2E), real-hooks discipline.
run-before-prod	KEEP	Surface→suite gate + 4-way failure discrimination; catalog templated.
nexor-brain	KEEP	Atomic-wiki methodology; paths already parameterized.
OPERATIONS		
webchat-incident	INTERNAL	Defense blind-spot disclosure — see page 1. Its EVAL.md (behavioral skill-eval methodology) is publishable later as a standalone write-up if wanted.
nexor-onboard	INTERNAL	Prod-DB operator tool with RLS-bypass key — see page 1.
SALES COLLATERAL		
proposal-builder	KEEP	Deliberately published (PR #2). Brand-coupled so low reuse for strangers, but harmless and demonstrates the HTML→PDF pipeline. Optional: generalize the template later.
INDUSTRY ONBOARDING		
real-estate	KEEP	Bank thresholds cited are public-domain banking info (CL market), not client data.
healthcare · saas-b2b · ecommerce	KEEP	Marquee vertical playbooks — the package's most reusable content.
education · fitness · hospitality	KEEP	Same.
generic	KEEP	Fallback playbook; anchors the vertical set.
THIRD-PARTY		
agent-browser	WITHHELD	Already excluded from the tarball (license: not ours to redistribute). Stays in-repo for internal agents.

Action plan (Part 1)

1. Add `query-catalog`, `quick-meeting`, `webchat-incident`, `nexor-onboard` to `nexorSkills.unpublished` + matching `files` negations. CI enforces the rest.
2. Split `data-analysis-playbook`: schema tables → `internal/schema.md`; keep hygiene rules in `SKILL.md`.
3. Scrub production percentages from `how-to-move-leads` frontmatter (keep the “most failure-prone family” framing).
4. Remove the four withdrawn skills from `marketplace.json` (the gate fails the build until this is done — by design).
5. Merge to `main` → CI publishes `@nexor/skills` with the final public set of **29 skills**.

The six primitives, and the ladder that orders them

Everything a client asks for lands on one of six primitives. The skill's core claim: **config beats code** — config is operator-visible, editable in the UI, and cannot crash; code is invisible to operators and yours to maintain forever. Descend the ladder only when the rung above cannot express the requirement.

#	PRIMITIVE	TRIGGER	CODE?	SCOPE	CONFIGURED IN
1	Agent + tools	Lead's message, mid-conversation	No	One lead, in-turn	Workflow config, <code>workflow_tools</code>
2	Rules (reminder / host / recontact)	Meeting lifecycle, staleness	No	One lead	Rules config (declarative, idempotent upserts)
3	Background job	<code>cron</code> · <code>event</code> · <code>api</code> (forced)	No	Filtered cohort	Job config: filters + steps + actions
4	Cloud function	A <code>lead.*</code> event	JS	One lead per event	Sandboxed worker, effects buffer
5	Scheduled function	<code>cron</code> + timezone	JS	Cohort (lookup query)	Sandboxed worker, effects buffer
6	Workflow transfer	Status with <code>transfer_config</code> reached	No	One lead	Status config; chain snapshot carried

1 Can the agent do it in conversation?

A status, a required field, a stage-gated tool, a prompt rule → **agent config**. "Notify us when budget > X" is usually just a tool call in-turn.

2 Is it a standard lifecycle nudge?

Booking confirmations, pre-meeting reminders (signed `delay_minutes` vs `event_start`), host notifications, stale-lead recontact → **rules**. Configure and stop.

3 Is it expressible as "filter leads → condition → action"?

Job step vocabulary: `evaluate_metadata` / `evaluate_field` / `evaluate_tag` (date-in-tag) → actions `send_message` (approved template, `then_status_key` for send-once), `set_lead_status`, `assign_workflow`, `workflow_transfer`, `enrich_lead` → **background job**. Always set `max_leads_per_cycle` (≤500) + `cooldown_minutes`; dry-run anything that messages humans.

4 Does it need custom logic?

Branching, computation, or an external API the job vocabulary can't express. Trigger grammar routes it: "**when** a lead ..." → **cloud function** (event, one lead). "**every** morning / Monday ..." → **scheduled function** (cron, cohort). A "when" that tolerates a day of latency across many leads → prefer the scheduled sweep (dry-run shows the whole cohort first).

5 Does the lead need a different agent?

Goal / persona / cadence / channel mix changes fundamentally → **transfer**. If both "agents" would share 80% of their prompt, it's one agent with more statuses.

Cloud vs scheduled functions — the semantics that decide designs

Both run tenant JS in an isolated sandbox and **buffer effects** instead of mutating directly. Every effect is scoped to the function's own client; a foreign lead id no-ops. One failed effect is recorded without aborting the rest. From there, the two types diverge on purpose:

Cloud function event · one lead

- **Triggers (6):** `lead.created`, `lead.updated`, `lead.status_changed`, `lead.assigned`, `lead.tag_added`, `lead.unsubscribed`
- **ctx:** the lead + the event; effects default to *that* lead
- **No chaining:** `assignLead` does *not* re-emit events — the recursion guard exists because `lead.updated` → `updateLead` → ∞. A design that needs chained events is a scheduled function.
- **Manual test runs are dry-run by default** — effects previewed, not applied

Use when: enrich a new lead via external API · recompute a score on field change · mirror a status into the client's CRM · react to an unsubscribe.

Scheduled function cron · cohort

- **Trigger:** `cron_expression` + timezone; cohort pre-resolved by a lookup query into `ctx.leads`; capped by `max_leads_per_cycle`
- **Every effect names its own lead** — there is no “the” lead
- **Chaining allowed:** `assignLead` intentionally re-emits `lead.assigned` / `lead.updated` — a sweep can assign and let a cloud function react per-lead
- **Manual runs always dry-run:** candidates found + effects previewed. Read that list before activating anything that messages people.

Use when: nightly re-scoring · weekly digests · “follow-up date arrived” sweeps · data hygiene passes.

Shared effects API (both types): `updateLead` · `updateMetadata` · `upsertLead` · `setLeadStatus` · `assignLead` · `assignToWorkflow` · `tagLead` · `deactivateWorkflowRun` · `updateEnvironmentVariable`

Transfers — the agent-to-agent contract

- **Fired by:** a status carrying `transfer_config` → `target_workflow_id`; or cohort-scale via a job/function action.
- **What the platform does:** cancels source cadence, deactivates the source run (`outcome='transferred'`), creates a *fresh* target run, reassigns the conversation thread, accumulates a **transfer chain** with a snapshot of source fields (multi-hop: C sees A's and B's fields).
- **Fields do not copy** into target field values — schemas differ by design; the target agent reads the chain snapshot.
- **First contact:** AI transition message when the messaging window is open (contextual handoff composed by the target agent), else the target's hot-contact template. Never also aim a “welcome” job at freshly-transferred leads — the handoff already speaks.
- **Fresh-run rule:** A→B→A is safe (no zombie runs) but every hop restarts cadence — if the “transfer” is a stage of the same conversation, it's a status, not a transfer.

The five anti-patterns the skill encodes: a function that a rule already was · an event function relying on its own re-trigger (the guard silently eats step 2) · cohort messaging without `then_status_key` /cooldown (re-sends every cycle) · transfers used as pipeline stages · activating cohort automations without reading the dry-run.

Use case → exact configuration

The worked mappings the skill ships as templates. Column three is deliberately concrete: the deliverable of the skill is a configuration, not a recommendation.

CLIENT ASKS FOR...	PRIMITIVE	EXACT CONFIGURATION
"After every completed meeting, send a satisfaction survey — once."	JOB	Cron daily · workflow filter on post-meeting status · action <code>send_message</code> (approved template) with <code>then_status_key: surveyed</code> . Send-once is <i>stateful</i> , never assumed.
"Remind the lead the day before and the morning of the meeting."	RULES	Two reminder rules on <code>event_start: delay_minutes -1440</code> and <code>-180</code> . Nothing else — not a cron, not a function.
"When a lead fills in company size, enrich from our API and score them."	CLOUD FN	<code>lead.updated</code> · guard on the field actually changing · external fetch · <code>updateMetadata</code> + <code>tagLead</code> . Code earned by the external call + computation.
"Every Monday, re-activate leads whose 'call me in N weeks' date arrived."	JOB → SCHED FN	First try config: <code>evaluate_field date_before_today</code> in a job. Only if the date needs computing: scheduled fn, cron <code>0 9 * * 1</code> , lookup on the parked status, <code>setLeadStatus</code> per lead.
"Once qualified, a different flow schedules the demo with its own cadence."	TRANSFER	<code>transfer_config</code> on the <code>qualified</code> status → target workflow. Target agent reads budget/need from the transfer chain; its cadence + prompt take over; AI transition handles the handoff message.
"When a lead unsubscribes anywhere, flag them in our CRM."	CLOUD FN	<code>lead.unsubscribed</code> → CRM HTTP call. An agent tool is wrong here: unsubscribes also happen outside conversations.
"Send this campaign once, when we say go."	JOB (API)	<code>trigger_type: api</code> + forced run. Stays visible, auditable, re-runnable — never a one-off script.
"One sales conversation with contacted / qualified / booked stages."	ONE AGENT	Statuses on a single workflow. Three agents here is the canonical mistake: every hop resets cadence and fragments history.

Where this lands

- **Skill shipped:** `skills/automation-architecture/SKILL.md` — registered in `marketplace.json` (agent-playbooks), passes the publish gate (34 on disk, 33 publishable).
- **Distribution:** next merge to `main` publishes it in `@nexor/skills`; the master-editor and dashboard agents receive it through the same sync as every other skill.
- **Next iteration (recommended):** a trigger eval for its description (the webchat-incident EVAL.md method — positives: "when should I use a scheduled function"; negative controls: pure prompt edits, single reminder asks), and a worked-example eval that feeds it three raw use cases and grades the configurations against this table.